МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом

Студентка гр. 4345 _____ Жилина А. Е.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2026

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до N−1, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.

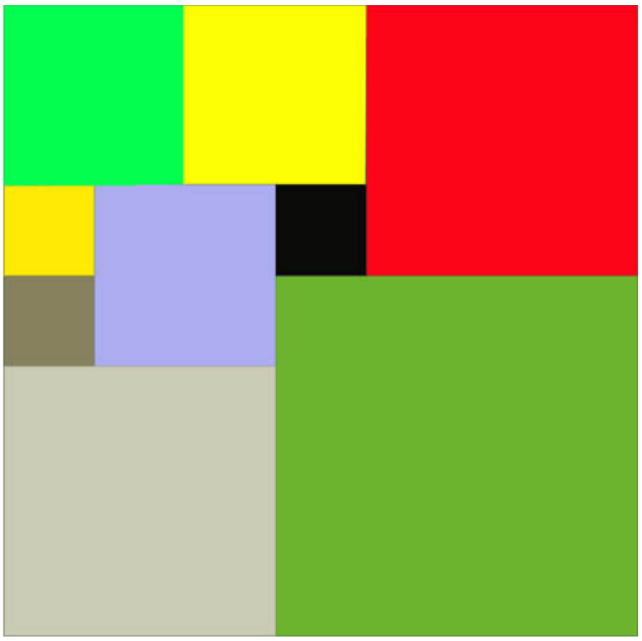

Рисунок 1 – пример расположения квадратов для тестового сучая

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные:

Размер столешницы - одно целое число N (2≤N≤20).

Выходные данные:

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа

x,y и w, задающие координаты левого верхнего угла ($1 \le x,y \le N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Индивидуальный вариант: 1р. Рекурсивный бэктрекинг. Выполнение на Stepik всех трёх заданий в разделе 2.

Описание алгоритма

Основная идея алгоритма заключается в комбинации рекурсивного бэктрекинга (поиска с возвратом) и алгоритма итеративного углубления, дополненных строгими математическими эвристиками.

В отличие от классического поиска в ширину или использования явного стека состояний, наш алгоритм не пытается найти все возможные разбиения, чтобы затем выбрать минимальное. Вместо этого вводится жесткое ограничение (лимит) на количество используемых квадратов. Алгоритм пытается найти решение ровно за L шагов. Если это не удается, лимит увеличивается на единицу. Благодаря этому, первое же найденное решение гарантированно является оптимальным, и программа немедленно завершает работу.

Для исключения дублирования состояний (симметрий) алгоритм строго детерминирован: он всегда ищет первую свободную клетку слева направо и сверху вниз. Найдя её, он пытается по очереди вписать туда квадраты начиная с максимального размера и постепенно уменьшая его вплоть до 1x1.

Сложность по времени: O(N^{L+2}) (в худшем случае)

Ограничение глубины дерева: В отличие от обычного бэктрекинга, где глубина рекурсии может достигать N^2, итеративное углубление ограничивает глубину переменной L (где L — текущий лимит квадратов).

Основной цикл: В худшем случае на каждом из L уровней рекурсии алгоритм может порождать до O(N) новых ветвей (перебор размеров квадрата). Это дает O(N^L) состояний.

Вспомогательные операции: На каждом шаге поиск пустой клетки занимает в худшем случае O(N^2), проверка границ и обновление матрицы доски также занимают O(N^2).

Общая теоретическая сложность: O($N^L * N^2$) = O(N^{L+2}). Поскольку в теории L может стремиться к N^2, сложность приближается к O(N^{N^2+2}).

Сложность по памяти: O(N^2):

Матрица столешницы N x N занимает O(N^2) памяти.

При переходе на новый уровень алгоритм не копирует доску, а лишь меняет значения в глобальном массиве, откатывая их при возврате (бэктрекинге).

Максимальная глубина стека вызовов рекурсии равна L (которое не превышает N^2). Вектор текущего решения также хранит не более N^2 элементов.

Итоговая сложность: O(N^2) + O(N^2) = O(N^2).

Оптимизация алгоритма

Несмотря на огромную асимптотику худшего случая, алгоритм включает оптимизации, которые значительно уменьшают дерево поиска:

1. Любое составное число N можно представить как произведение его наименьшего простого делителя d и масштабного коэффициента k (где N = d * k). Математически доказано, что оптимальное (содержащее минимальное количество элементов) разбиение квадрата со стороной N структурно полностью идентично разбиению квадрата со стороной d. Разница заключается лишь в том, что размер каждого внутреннего элемента увеличивается ровно в k раз. Таким образом, минимальное количество квадратов, необходимых для заполнения столешницы размером 15x15, будет точно таким же, как и для столешницы 3x3.

Благодаря этому свойству, алгоритму не нужно перебирать огромное количество комбинаций для исходной большой доски. Вместо этого программа определяет минимальный делитель d и запускает поиск решения для сетки d x d. В результате вычислительная сложность падает с O(N^{N^2+2}) до O(d^{d^2+2}). После того как алгоритм за доли секунды находит оптимальную конфигурацию для малого квадрата, координаты левых верхних углов и длины сторон всех найденных элементов просто умножаются на масштабный коэффициент k, выдавая готовый ответ для исходного числа N.

2. Для простых N алгоритм опирается на строгую геометрическую закономерность: оптимальное разбиение всегда начинается с квадрата размером N/2 с округлением вверх в левом верхнем углу и двух квадратов размером N/2 с округлением вниз по бокам от него. Принудительная постановка этих трех фигур до начала перебора мгновенно покрывает около 75% площади столешницы. Это отсекает огромное количество заведомо тупиковых ветвей дерева поиска, оставляя рекурсивному бэктрекингу лишь небольшую L-образную полосу в нижнем правом углу.

Способ хранения частичных решений

Для хранения промежуточных состояний (частичных решений) мы не создаем копии доски на каждом шаге, так как это потребляло бы слишком много памяти. Вместо этого используется один глобальный двумерный массив board и вектор current_solution. В массиве нулями отмечаются пустые клетки столешницы, а единицами — занятые. Вектор же работает как стек: он просто хранит список уже поставленных квадратов (их координаты и размер) для текущей ветки перебора.

Когда алгоритм ставит новый квадрат, он отмечает нужные клетки в массиве board единицами и добавляет параметры квадрата в конец вектора current_solution. Если алгоритм заходит в тупик и нужно сделать шаг назад (бэктрекинг), он просто удаляет последний квадрат из вектора и возвращает нули в соответствующие клетки массива. Если вся доска успешно заполняется, содержимое current_solution копируется в вектор лучшего решения best_solution.

Описание рекурсивной функции (бэктрекинг)

Основа алгоритма — рекурсивная функция *bool solve(int current_count, int filled_area)*. Её задача — найти первую свободную клетку на столешнице (поиск идет строго слева направо и сверху вниз) и попытаться вписать в неё квадрат. Программа перебирает размеры квадратов от максимально возможного до 1x1 и рекурсивно вызывает саму себя, чтобы заполнить оставшееся место.

Функция принимает два аргумента: current_count (количество уже поставленных обрезков) и filled_area (площадь, которую они заняли). Мы передаем площадь специально, чтобы мгновенно проверять, заполнилась ли столешница, а не сканировать для этого весь массив каждый раз. Функция возвращает логическое значение (bool): true означает, что мы нашли верное решение и замостили всю доску, а false означает тупик (квадраты не подходят или превышен лимит количества). Получив false, алгоритм делает шаг назад и пробует квадрат меньшего размера.

Описание функций и структур данных

Структуры данных и глобальные переменные:

struct Square — Структура, описывающая один обрезок доски (квадрат). Содержит три целочисленных поля: x и y — координаты левого верхнего угла квадрата и size — длина стороны данного квадрата.

int board[40][40] — Глобальный двумерный массив, представляющий текущее состояние столешницы. В отличие от хранения состояний в стеке, мы используем одну глобальную доску. Значение 0 означает, что клетка пуста, значение 1 — клетка занята. Это позволяет обращаться к клеткам за время O(1) без накладных расходов на копирование матриц.

int LIMIT — Глобальная переменная, хранящая максимально допустимое количество квадратов для текущей итерации итеративного углубления.

std::vector<Square> current_solution — Вектор, выполняющий роль стека для текущего пути в дереве поиска. Он хранит квадраты, которые уже размещены на доске в текущей ветке рекурсии.

std::vector<Square> best_solution — Вектор, в который копируется current_solution в тот момент, когда алгоритм заполняет всю доску (находит решение).

Функции алгоритма:

int get_min_factor(int n) — Вспомогательная математическая функция для оптимизации масштабирования. Принимает размер исходной столешницы n. Через цикл перебирает числа от 2 до sqrt(n). Возвращает наименьший делитель числа. Если число простое — возвращает само число.

bool solve(int current_count, int filled_area) — Рекурсивная функция, реализующая основной алгоритм бэктрекинга (поиска с возвратом). Возвращает true, если решение найдено, и false, если ветка тупиковая.

Аргументы: current_count (текущее количество поставленных квадратов) и filled_area (общая площадь столешницы, уже покрытая обрезками). Передача площади позволяет проверять заполненность доски за O(1) вместо обхода всей матрицы.

Базовые случаи: В начале функция проверяет два условия. Если filled_area == N * N, доска успешно заполнена: текущее решение сохраняется в best_solution, и функция возвращает true. Если current_count == LIMIT, достигнут предел глубины поиска для текущей итерации итеративного углубления, происходит отсечение ветки (возврат false).

Поиск свободной клетки: С помощью двойного цикла функция сканирует массив board строго сверху вниз и слева направо в поисках первой клетки со значением 0. Координаты этой клетки записываются в tx и ty.

Определение максимального размера: Найдя пустую клетку, алгоритм с помощью цикла while вычисляет max_s — максимально возможный размер квадрата, который можно поставить в эту точку, не выходя за границы доски N и не пересекаясь с уже стоящими квадратами (проверяя только правый и нижний края растущего квадрата).

Рекурсивный перебор и бэктрекинг: Запускается цикл от max_s до 1. Для каждого размера s алгоритм ставит квадрат (заполняет ячейки board единицами), добавляет его параметры в current_solution и рекурсивно вызывает solve(current_count + 1, filled_area + s * s). Если вызов вернул true, успех пробрасывается вверх по стеку. Если false — выполняется бэктрекинг: квадрат удаляется из current_solution, а соответствующие ячейки массива board снова заполняются нулями, после чего цикл пробует квадрат меньшего размера.

int main() — Главная функция, управляющая логикой программы.

Считывает исходный размер N.

Вызывает get_min_factor, чтобы найти делитель d и масштабный коэффициент scale. Размер доски переопределяется как N = d (оптимизация масштабирования).

Предрасстановка: Если N > 2, вычисляются размеры базовых квадратов K = N / 2 + 1 и n_k = N - K. Доска очищается, на нее ставятся три этих квадрата в верхний левый, верхний правый и нижний левый углы, вычисляется их суммарная площадь.

Итеративное углубление: Запускается цикл, где LIMIT постепенно увеличивается от 6 до N^2 . Внутри цикла вызывается solve(3, filled_area). Использование этого цикла гарантирует, что первым алгоритм найдет решение с минимально возможным количеством квадратов. Как только solve возвращает true, цикл прерывается.

В конце функция проходит по best_solution, умножает координаты и размеры каждого квадрата на scale (прибавляя 1 для корректной индексации вывода) и выводит результат в консоль.

Тестирование

Результаты тестирования представлены в табл. 1.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные	Комментарии
1.	7	9 1 1 4 5 1 3 1 5 3 5 4 2 7 4 1 4 5 1 7 5 1 4 6 2 6 6 2	Верный результат
2.	6	4 1 1 3 4 1 3 1 4 3 4 4 3	Верный результат
3.	39	6 1 1 26 27 1 13 1 27 13 27 14 13 14 27 13 27 27 13	Верный результат
4.	37	15 1 1 19 20 1 18 1 20 18	Верный результат

		20 19 2 22 19 5 27 19 11 19 20 1 19 21 3 19 24 8 27 30 3 30 30 8 19 32 6 25 32 1 26 32 1 25 33 5	

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>

using namespace std;

// Структура для хранения информации о квадрате
struct Square {
    int x, y, size;
};

int N;
int board[40][40]; // Двумерный массив (доска)
int LIMIT;
vector<Square> current_solution;
vector<Square> best_solution;

// Файл логов
ofstream log_file;



// Рекурсивная функция поиска
bool solve(int current_count, int filled_area) {
    string ind = string(current_count * 4, ' ');
    log_file << ind << "--> [ВХОД] Уровень: " << current_count << " |
Заполненная площадь: " << filled_area << "/" << N * N << "\n";

    // Вся площадь столешницы заполнена, ответ найден
    if (filled_area == N * N) {
        log_file << ind << "    [УСПЕХ] Вся площадь заполнена! Решение
найдено.\n";
        best_solution = current_solution;
        return true;
    }
```

```
    // Исчерпан лимит - отсечение ветки
    if (current_count == LIMIT) {
        log_file << ind << "    [ОТСЕЧЕНИЕ] Достигнут лимит в " << LIMIT
<< " квадратов. Возврат.\n";
        return false;
    }

    // Ищем первую свободную ячейку (сверху вниз, слева направо)
    int tx = -1, ty = -1;
    for (int y = 0; y < N; ++y) {
        for (int x = 0; x < N; ++x) {
            if (board[y][x] == 0) {
                ty = y; tx = x;
                break;
            }
        }
        if (ty != -1) break;
    }

    log_file << ind << "    [ПОИСК] Первая свободная ячейка: x=" << tx
<< ", y=" << ty << "\n";

    // Ищем максимально возможный размер квадрата для этой ячейки
    int max_s = 1;
    while (ty + max_s < N && tx + max_s < N) {
        bool ok = true;
        for (int i = 0; i <= max_s; ++i) {
            if (board[ty + max_s][tx + i] != 0 || board[ty + i][tx +
max_s] != 0) {
                ok = false;
                break;
            }
        }
        if (!ok) break;
        max_s++;
    }

    log_file << ind << "            Максимальный размер квадрата для
этой ячейки: " << max_s << "x" << max_s << "\n";
```

```
    // Перебираем размеры от большего к меньшему
    for (int s = max_s; s >= 1; --s) {
        log_file << ind << "    [ДЕЙСТВИЕ] Ставим квадрат " << s << "x"
<< s << " в координаты (x=" << tx << ", y=" << ty << ")\n";

        // Ставим квадрат (заполняем массив 1)
        for (int y = ty; y < ty + s; ++y)
            for (int x = tx; x < tx + s; ++x)
                board[y][x] = 1;

        current_solution.push_back({tx, ty, s});

        // Углубляемся. Если нашли решение - возвращаем его
        if (solve(current_count + 1, filled_area + s * s)) {
            return true;
        }

        // Иначе - бэктрекинг (убираем квадрат и пробуем другой)
        log_file << ind << "     [БЭКТРЕКИНГ] Откат. Убираем квадрат "
<< s << "x" << s << " из (x=" << tx << ", y=" << ty << ")\n";
        current_solution.pop_back();
        for (int y = ty; y < ty + s; ++y)
            for (int x = tx; x < tx + s; ++x)
                board[y][x] = 0;
    }

    // Если ни один размер квадрата не подошёл, возвращаем false
    log_file << ind << "<-- [ВЫХОД] Ни один размер для ячейки (" << tx
<< ", " << ty << ") не привёл к решению. Идём назад.\n";
    return false;
}

// Вспомогательная функция для нахождения минимального делителя
int get_min_factor(int n) {
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) return i;
    }
    return n;
```

```
}

int main() {
    // Открываем файл для записи логов
    log_file.open("log.txt");
    if (!log_file.is_open()) {
        cerr << "Не удалось открыть файл log.txt для записи!\n";
        return 1;
    }

    int original_N;
    if (!(cin >> original_N)) return 0;

    log_file << "=== СТАРТ ПРОГРАММЫ ===\n";
    log_file << "Исходный размер столешницы: N = " << original_N << "\n";

    // Масштабирование для составных чисел
    int d = get_min_factor(original_N);
    int scale = original_N / d;

    N = d; // Ищем решение только для наименьшего делителя
    log_file << "Минимальный делитель: " << d << ". Масштабный множитель:
" << scale << "\n";
    if (original_N != N) {
        log_file << "Ищем решение для упрощенной доски N = " << N <<
"\n";
    }

    if (N == 2) {
        log_file << "Сработал базовый случай для N=2 (или четных
чисел).\n";
        best_solution = { {0, 0, 1}, {1, 0, 1}, {0, 1, 1}, {1, 1, 1} };
    } else {
        int K = N / 2 + 1; // Размер углового квадрата
        int n_k = N - K;   // Размер двух соседних квадратов

        // Итеративное углубление. Минимум для простых чисел начинается
с 6.
        for (LIMIT = 6; LIMIT <= N * N; ++LIMIT) {
```

```
            log_file << "\n----------------------------------------\n";
            log_file << "ЗАПУСК ПОИСКА С ЛИМИТОМ КВАДРАТОВ = " << LIMIT
<< "\n";
            log_file << "----------------------------------------\n";

            // Сбрасываем доску и текущее решение
            for (int i = 0; i < N; ++i)
                for (int j = 0; j < N; ++j)
                    board[i][j] = 0;
            current_solution.clear();

            log_file << "Расстановка 3 базовых квадратов:\n";
            // 1. Ставим левый верхний квадрат
            for (int y = 0; y < K; ++y)
                for (int x = 0; x < K; ++x)
                    board[y][x] = 1;
            current_solution.push_back({0, 0, K});
            log_file << "  - Квадрат " << K << "x" << K << " в (0, 0)\n";

            // 2. Ставим правый верхний квадрат
            for (int y = 0; y < n_k; ++y)
                for (int x = K; x < N; ++x)
                    board[y][x] = 1;
            current_solution.push_back({K, 0, n_k});
            log_file << "  - Квадрат " << n_k << "x" << n_k << " в ("
<< K << ", 0)\n";

            // 3. Ставим левый нижний квадрат
            for (int y = K; y < N; ++y)
                for (int x = 0; x < n_k; ++x)
                    board[y][x] = 1;
            current_solution.push_back({0, K, n_k});
            log_file << "  - Квадрат " << n_k << "x" << n_k << " в (0,
" << K << ")\n";

            int filled_area = K * K + 2 * n_k * n_k;

            // Запускаем поиск (3 квадрата уже поставлены)
            if (solve(3, filled_area)) {
```

```
                log_file << "\n>>> РЕШЕНИЕ НАЙДЕНО ПРИ ЛИМИТЕ = " <<
LIMIT << " <<<\n";
                break;
            }
        }
    }

    log_file.close();

    // Вывод ответа
    cout << best_solution.size() << "\n";
    for (const auto& sq : best_solution) {
        cout << sq.x * scale + 1 << " "
             << sq.y * scale + 1 << " "
             << sq.size * scale << "\n";
    }

    return 0;
}
```